

Documento Requisiti — Ruffino Ops (PRD)

Stato: Documento vivente, riallineato allo stato corrente dell'applicazione (16/07/2026). **Versione:** 4.0 — Multi-sede, Magazzino, Pagamenti/acconti, sincronizzazione Google Calendar (export+import), backup notturno su Google Drive, Fatture in Cloud, WhatsApp, notifiche personalizzate v2, timeline ordine con note, migrazione dati 2026. **Riferimento implementativo:** repository `infissi-ops-app`. Il presente PRD descrive il comportamento atteso del software così come è implementato; ogni divergenza riscontrata nel codice va trattata come bug.

0. Convenzioni del documento

- **MUST / DEVE** — requisito obbligatorio.
 - **SHOULD / DOVREBBE** — requisito fortemente consigliato.
 - **MAY / PUÒ** — opzione facoltativa.
 - Gli identificatori `in_questo_stile` sono valori interni (enum, chiavi di database). Le etichette UI in italiano sono indicate fra virgolette.
 - I numeri di sezione sono stabili; le sotto-sezioni possono crescere.
-

1. Visione del prodotto

Ruffino Ops è lo strumento operativo centrale di **Ruffino Immobiliare S.R.L.** Collega ufficio, laboratorio di produzione e cantiere, accompagnando ogni cliente dalla prima richiesta fino alla garanzia post-vendita. Non è un semplice database: è un **assistente proattivo** che ricorda le scadenze in base alla priorità delle commesse, blocca i passaggi di stato senza i documenti richiesti, e fornisce viste operative (Board, Calendario, Classifica) pensate per le persone reali che le useranno tutti i giorni.

Pilastri:

1. **Una commessa = un percorso tracciato.** Stato, documenti, interventi, anomalie e firme convivono in un unico fascicolo.
 2. **Niente dato perso.** Le commesse rifiutate dal cliente vengono **archivate**, non cancellate; in qualsiasi momento si possono ripristinare con stato e file invariati.
 3. **Sicurezza by default.** Autenticazione obbligatoria su ogni endpoint; password hashate; gate documentale lato server.
 4. **Operatività prima dell'eleganza.** Pagine come Calendario e Board mostrano in faccia all'utente le informazioni che gli servono (nome, cognome, indirizzo lavoro, telefono cliccabile).
-

2. Architettura tecnica (sintesi)

- **Frontend.** React 19 + Vite + Wouter (routing) + tRPC v10 (client) + React Query (caching) + Tailwind + shadcn/ui + lucide-react + sonner (toast) + jsPDF/jspdf-autotable per i PDF dei preventivatori.

- **Backend.** Node + Express + tRPC v10. Persistenza in `kv_store` (Postgres JSONB) tramite un piccolo layer `persistedStore` con save debounciato, retry su errori transienti, recovery in background.
 - **Autenticazione.** Locale via email/password con JWT firmato (jose, HS256, TTL 7 giorni) + cookie http-Only. Sessione server-side cacheata in memoria con eviction periodica.
 - **Sicurezza.** Tutti gli endpoint business sono `protectedProcedure` (utente loggato obbligatorio); le mutazioni su `utenti` e l'intero router `backup / fattureInCloud` sono `adminProcedure` (ruolo direzione). Header `X-Content-Type-Options`, `X-Frame-Options=SAMEORIGIN`, `Referrer-Policy`, HSTS in produzione. Upload con allowlist mimeType + validazione reale del payload base64. CSRF same-origin check su `/api/trpc`. `trust proxy` abilitato (deploy dietro Railway).
 - **Scheduler interni.** Backup notturno Google Drive (00:00 Europe/Rome, `setTimeout` ri-armato), sync Fatture in Cloud (ogni 6 h quando abilitato).
 - **PDF.** jsPDF + jspdf-autotable sia client-side (preventivatori, scheda cliente) sia server-side (scheda cliente nel backup).
 - **Storage file.** Documenti caricati come base64 nel KV store, separati per dominio (`preventivi_documents`, `ticket_allegati`). Cap per-file 10 MB.
-

3. Autenticazione e sicurezza

3.1 Login

- Schermata `LoginPage` con email + password.
- Login solo per utenti `attivo === true`.
- Confronto password tramite `verifyPassword` (scrypt) — accetta anche password legacy in chiaro durante una migrazione trasparente.
- **Rate limit per email:** dopo 5 tentativi falliti in 15 minuti l'account è bloccato fino allo scadere della finestra. Login riuscito → reset del contatore.
- Messaggio di errore generico ("Email o password non validi") per non rivelare se l'utente esiste.

3.2 Sessione

- JWT firmato HS256, claim: `sub` (id utente), `email`, `name`, `role`, `ruolo`, `ruoli`. Esp. 7 giorni.
- Cookie `httpOnly`, `secure` su HTTPS, `sameSite none` su https / `lax` su http locale.
- Cache server-side `Map<token, { user, expMs }>` con eviction lazy e sweep orario (`setInterval(...).unref()`).
- **Logout** cancella sia il cookie sia l'entry della cache server.

3.3 Hashing password

- Algoritmo: **scrypt** (modulo `crypto` di Node, nessuna dipendenza esterna).
- Formato di archiviazione: `scrypt$<saltHex>$<hashHex>` (salt 16 byte, key 64 byte).
- `verifyPassword` è constant-time tramite `timingSafeEqual`.
- Le password create/aggiornate dalla UI sono sempre hashate.
- Le password legacy in chiaro residue nel DB vengono **automaticamente upgradeate a hash al primo boot successivo** all'aggiornamento (vedi `utenti.onLoad`).

3.4 Segreto JWT

- In **produzione** la variabile d'ambiente `JWT_SECRET` è OBBLIGATORIA: in sua assenza il server fa throw all'avvio.
- In dev/test, in mancanza di `JWT_SECRET`, viene usato un fallback hard-coded; **non valido per produzione**.

3.5 Header di sicurezza

Su ogni risposta HTTP:

- `X-Content-Type-Options: nosniff`
- `X-Frame-Options: SAMEORIGIN`
- `Referrer-Policy: strict-origin-when-cross-origin`
- `X-DNS-Prefetch-Control: off`
- `X-Permitted-Cross-Domain-Policies: none`
- In produzione: `Strict-Transport-Security: max-age=15552000; includeSubDomains`
- CSP **non** impostata di default (richiede tuning con Vite + Maps proxy + blob preview) — tracciata come follow-up.

3.6 Esposizione API

- Endpoint tRPC montati su `/api/trpc`.
- Tutti i router business utilizzano `protectedProcedure`. Le sole eccezioni pubbliche sono: `auth.me`, `auth.login`, `auth.logout`, `system.health`.
- `utenti.create`, `utenti.update`, `utenti.delete` richiedono `adminProcedure` (= ruolo `direzione`).

3.7 Protezioni aggiuntive (v4)

- **Hash versionato.** Formato `scrypt$<N>$<saltHex>$<hashHex>` con `N=32768`; verifica retro-compatibile con il legacy `scrypt$salt$hash` (`N=16384`) e con password in chiaro (upgrade automatico al boot).
- **CSRF.** Su `/api/trpc` le richieste con header `Origin` diverso dall'`Host` vengono rifiutate (403). Richieste server-to-server senza `Origin` ammesse.
- **SSRF guard (calendari esterni).** Gli URL iCal inseriti dagli operatori sono validati (solo https; vietati `localhost/`, `.local/`, `.internal/` IP privati RFC1918/link-local/IPv6 raw) sia all'inserimento sia a ogni fetch.
- **Segreti mascherati in UI.** Gli indirizzi iCal privati (bearer secret) e i token Fatture in Cloud sono ritornati mascherati dalle list/status API.
- **Isolamento sede.** Ogni query/mutation è vincolata alla sede attiva (`ctx.sedeId`); guardie `assertSedeScope` su tutte le entità (vedi §34).

3.8 Operatività residua a carico del titolare (non risolvibile via codice)

- Rotazione manuale delle password committate in git history prima della migrazione a hash.
 - Pulizia dello storico Git da quelle password (BFG / `git filter-repo` + force-push).
-

4. Ruoli, permessi e gating

4.1 Set di ruoli

- `direzione` — ammessi accesso completo + gestione utenti + sezioni gated (Squadre, Garanzie, Fornitori, Produzione, Utenti, Integrazioni avanzate).
- `amministrazione` — focus su fatturazione, finiture, saldo.
- `commerciale` — venditori; partecipano alla **Classifica**.
- `tecnico_rilievi` — rilievi e misure.
- `squadra_posa` — esecuzione in cantiere.
- `post_vendita` — ticket, reclami, rifacimenti.
- `ordini` — ordini fornitori.

Ogni utente ha `ruoli: string[]` (1–3 valori). Il campo legacy `ruolo` continua a contenere il ruolo primario per retro-compatibilità.

4.2 Mapping `role` legacy

- `role = "admin"` quando in `ruoli` è presente `direzione`. Altrimenti `user`.

4.3 Gating

- **Lato server.** `protectedProcedure` su tutto il business; `adminProcedure` su mutazioni `utenti` e `system.notifyOwner`.
- **Lato client.** Componente `RequireDirezio` su rotte `Garanzie`, `Squadre`, `Fornitori`, `Produzione`, `Utenti`. Le voci di sidebar corrispondenti sono filtrate con il flag `direzioneOnly`.

5. Anagrafica Cliente

5.1 Tipi cliente

`privato` | `azienda` | `condominio` | `ente_pubblico`. Icone in UI: `User`, `Building2`, `Home`, `Landmark`.

5.2 Campi del Cliente

Anagrafica:

- **Nome** (obbligatorio).
- **Cognome** (obbligatorio).
- **Tipo** (vedi 5.1; default `privato`).
- **Codice fiscale, partita IVA**.

Doppio indirizzo:

- **Indirizzo di residenza** (`indirizzo`, `citta`, `cap`) — usato dall'amministrazione per la fatturazione.
- **Indirizzo dei lavori** (`indirizzoLavoro`, `cittaLavoro`, `capLavoro`) — usato dalle commesse per cantiere, calendario, mappe.

- In fase di creazione/modifica il form propone uno **switch "stesso della residenza"** che copia automaticamente i campi residenza → lavoro al salvataggio.
- La scheda cliente espone entrambi gli indirizzi con badge distintivo "Residenza" / "Lavoro".

Contatti: **telefono, email**.

Dati fiscali e amministrativi:

- **Detrazione** (switch). Quando attivata RICHIEDE `tipoDetrazione`.
- **Tipo detrazione**: `ecobonus` | `ristrutturazione` (obbligatorio se `detrazione === true`).
- **Finanziamento** (switch).
- **Pratica edilizia**: `nessuna` | `cil` | `cila` | `scia`.

Referenti (lista multipla):

- Campi per referente: nome, ruolo (`cliente_finale` | `architetto` | `direttore_lavori` | `amministratore` | `altro`), telefono, email.

Operativi:

- **Note** libere.
- **Assegnato a** (utente). Default: utente corrente; eredita su nuove commesse linkate al cliente.

5.3 Comportamenti del Cliente

- Lista clienti con ricerca testuale, filtro per tipo, filtro "solo le mie".
- Scheda cliente con tabs: **Commesse, Interventi, Ticket, Garanzie** — ognuno ha un pulsante **+** per creare l'entità collegata.
- **Cascade su update**. Modificando nome/cognome o un campo di contatto/indirizzo del cliente, il server propaga il valore a tutte le commesse linkate (solo dove il valore della commessa coincideva con il valore precedente del cliente, così le override manuali non vengono sovrascritte).
- Eliminazione cliente disponibile (con conferma) ma SCONSIGLIATA in presenza di commesse linkate.
- **Scheda PDF** stampabile dall'header (vedi §42) e bottone **WhatsApp** accanto al telefono (vedi §41).
- Ogni cliente appartiene a una sede (`sedeId`, vedi §34).

6. Commesse

6.1 Identificazione

- **Codice automatico** `COM-{ANNO}-{N}` con N progressivo a 3 cifre (es. `COM-2026-001`).
- Lo ZeroPadding è coerente per anno; al cambio anno il contatore può ripartire da 1.

6.2 Campi

- **clientId** (FK), **cliente** (display name denormalizzato, sincronizzato da `cliente.update`).
- **indirizzo, città, telefono, email** — inizialmente derivati dall' `indirizzoLavoro` del cliente; sovrascrivibili a livello di commessa.
- **stato** — vedi 7.1.
- **priorita**: `bassa` | `media` | `alta` | `urgente`.
- **squadraId** (FK opzionale).

- **dataApertura** (auto).
- **consegnaIndicativa**: enum "30" | "60" | "90" (giorni dal preventivo).
- **dataConsegnaIndicativa**: data libera ISO. **Mutuamente esclusiva** con `consegnaIndicativa`: salvando una delle due l'altra viene azzerata.
- **dataConsegnaConfermata**: data definitiva impostata al passaggio in `produzione`.
- **dataChiusura**: impostata automaticamente al passaggio in `archiviata` (chiusura del flusso).
- **note** libere.
- **importoTotale**: totale pattuito (€, opzionale). **importoIncassato**: derivato — somma del registro `pagamenti` (vedi §37); non modificabile direttamente dalla UI.
- **pagamenti**: registro acconti embedded (vedi §37). Escluso da `commesse.list` come `prodotti`.
- **sedeld**: sede proprietaria (vedi §34).
- **prodotti**: array di prodotti desiderati (nome, tipologia, quantità, dimensioni, note). NON viene incluso nella risposta di `commesse.list` per alleggerire i payload; viene ritornato da `commesse.byId`.
- **assegnatoA** (FK utente). Modificabile dalla scheda commessa.
- **createdBy** (FK utente).
- **createdAt**, **updatedAt**.
- **archivedAt**: timestamp ISO; `null` finché la commessa non è in soft-archive (vedi §24).

6.3 Consegna indicativa — selezione

La UI offre quattro opzioni nel select **Consegna indicativa**:

1. `+30 giorni`
2. `+60 giorni` (*default*)
3. `+90 giorni`
4. **Data da calendario...** → mostra un date picker, popola `dataConsegnaIndicativa` e svuota `consegnaIndicativa`.

L'header della commessa mostra in ordine di priorità:

1. `dataConsegnaConfermata` (etichetta "Data consegna prevista").
2. `dataConsegnaIndicativa` (etichetta "Consegna indicativa", formato `gg/mm/aaaa`).
3. `consegnaIndicativa` (etichetta "Consegna indicativa: +N giorni").

6.4 Trigger Produzione

Quando una commessa entra nello stato `produzione`:

1. Sulla scheda compare un banner ambra "Commessa in produzione" con il pulsante "Aggiorna data consegna".
2. Al click si apre un dialog con date picker per la **Data di Consegna Prevista** definitiva.
3. Il salvataggio popola `dataConsegnaConfermata` (visibile in header, card Kanban, lista in Dashboard).
4. Finché `dataConsegnaConfermata` è vuota, la card del Kanban resta evidenziata con anello ambra e contatore "Consegne da confermare" nel KPI bar.

6.5 Modifica

- Dialog **"Modifica commessa"** consente di modificare contemporaneamente:
 - Anagrafica cliente collegata (nome, cognome, CF, P.IVA, CAP, telefono, email, indirizzo, città) — sincronizzata via `clienti.update` con cascade.

- Priorità.
- **Utente assegnato** (SearchSelect sugli utenti, opzione "Non assegnata").
- Consegna indicativa (preset o data libera).
- Note.
- Nell'header sono visibili pill: indirizzo, telefono, email, **Assegnata a: Nome** (lookup utente).

6.6 Eliminazione vs Archiviazione

- **Archivia** — operazione consigliata se il cliente non procede. Non distrugge nulla (vedi §24).
- **Elimina** — operazione distruttiva. Conferma esplicita. Dovrebbe essere usata solo per errori di inserimento.

6.7 Lista commesse

- Filtri: search testuale (codice, cliente, città), stato, clienteld, assegnatoA, scope `archived = exclude | only | all` (default `exclude`).
- Ordinata per `createdAt desc`.
- Risposta **non include** `prodotti` (ottimizzazione bandwidth/render).

7. State machine delle commesse

7.1 Stati

Ordine canonico (`STATI_COMMESSA`):

1. `preventivo`
2. `misure_esecutive`
3. `aggiornamento_contratto`
4. `fatture_pagamento`
5. `da_ordinare`
6. `produzione`
7. `ordini_ultimazione` (*richiesta secondo acconto*)
8. `attesa_posa`
9. `finiture_saldo`
10. `interventi_regolazioni`
11. `archiviata` (*chiusura del flusso — distinta dal soft-archive di §24*)

7.2 Transizioni valide

- Transizioni **in avanti** consentite di **una posizione** (es. `da_ordinare` → `produzione`).
- Transizioni **all'indietro** consentite di **una posizione**.
- Salti multipli **vietati** lato server (`validateTransizione`).
- Stato `archiviata` raggiungibile solo dal predecessore `interventi_regolazioni`.

7.3 Soft-archive (orthogonal)

- `archivedAt` è ortogonale a `stato`. Una commessa può essere soft-archiviata in qualsiasi stato (vedi §24).

- Il soft-archive **non** modifica lo stato corrente.

7.4 Avanzamento via UI

- **Scheda commessa**: pulsante "Avanza" che propone lo stato successivo. Disabilitato se manca un documento richiesto (vedi §9), eccetto bypass con "Procedi comunque".
- **Board Kanban**: ogni card ha frecce **Indietro** / **Avanza**. Le frecce attraversano sempre la stessa state machine (controlli server identici).

7.5 Stati e gate documentale (vedi anche §9)

- preventivo → necessita preventivo o contratto.
 - misure_esecutive → misure.
 - aggiornamento_contratto → contratto.
 - fatture_pagamento → fattura.
 - da_ordinare → ordine o conferma_ordine.
 - produzione → nessun documento richiesto (gated da dataConsegnaConfermata).
 - ordini_ultimazione → saldo o fattura.
 - attesa_posa → ddt_consegna.
 - finiture_saldo → ddt_posa.
 - interventi_regolazioni → ddt_finale.
 - archiviata → nessun documento richiesto.
-

8. Documenti commessa (Preventivi/Contratti)

8.1 Tipi documento

preventivo, contratto, misure, fattura, ordine, conferma_ordine, ddt_consegna, ddt_posa, ddt_finale, saldo, foto, altro.

8.2 Storage e schema

- Persistito in `preventivi_documenti` (kv_store JSONB).
- Per ogni documento: `id`, `commessaId`, `nome`, `tipo`, `mimeType`, `size`, `dataBase64`, `note`, `stato`, `AtUpload`, `createdBy`, `createdAt`.
- Lista `byCommessa` strippa `dataBase64` e restituisce un flag `hasData`. Solo `byId` ritorna il contenuto.

8.3 Upload — controlli

- **MimeType allowlist** (stored XSS hardening):
 - `application/pdf`
 - `image/jpeg`, `image/png`, `image/gif`, `image/webp`, `image/heic`, `image/heif`
 - `application/msword`, `application/vnd.openxmlformats-officedocument.wordprocessingml.document`
 - `application/vnd.ms-excel`, `application/vnd.openxmlformats-officedocument.spreadsheetml.sheet`
 - **Esclusi esplicitamente** `text/html` e `image/svg+xml`.

- **Dimensione:** validata sul payload reale (lunghezza base64 decodificata, NON il campo `size` lato client). Cap 10 MB.
- Il `size` archiviato è quello calcolato dal server.

8.4 Auto-rename

- Se `keepNome === false`, il file in upload viene rinominato secondo `renameForStato({stato, cliente, originalName})` (es. "Misure esecutive Mario Rossi.pdf").
- Se `keepNome === true` (usato dai preventivatori), il nome viene preservato e solo dedupato.
- Disambiguazione automatica: se il nome esiste già per la stessa commessa, viene appeso `(2)`, `(3)`, ecc.

8.5 Anteprima e download

- Anteprima inline in `<iframe>` per PDF (URL `blob:` derivato dal base64) o in `<img>` con zoom/rotate per immagini.
- Download via `<a download>`.
- Invio email via `mailto:` con corpo precompilato (no upload server-side dell'allegato).

8.6 Eliminazione

- Soft delete NON previsto. La cancellazione è definitiva.

8.7 Allegati ticket

- Pattern identico (router `ticketAllegati`), stesso schema base64, stessa allowlist mime, stesso size check, cap 10 MB.
- Cancellando un ticket si cancellano in cascata i suoi allegati (`deleteAllegatiByTicket`).

9. Doc gate (gate documentale)

9.1 Regola

- Una transizione **in avanti** verifica che esista almeno un documento con uno dei tipi richiesti dallo stato CORRENTE (`REQUIRED_DOC_TIPI_PER_STATO`).
- Conta solo se il documento è stato caricato **mentre la commessa era in quello stato** (campo `statoAtUpload`), così un preventivo non può soddisfare un gate diverso.
- Per i documenti legacy senza `statoAtUpload`, fallback permissivo: il tipo è sufficiente.

9.2 Stati daily reminder

Per gli stati `aggiornamento_contratto`, `fatture_pagamento`, `da_ordinare` viene generata anche una notifica giornaliera (vedi §27.3) anche oltre la soglia di priorità.

9.3 Bypass "Procedi comunque"

- **Server.** `commesse.update` accetta un flag `force: boolean`. Se assente, il gate è bloccante; in caso di mancanza documenti il server lancia un errore con prefisso `DOC_GATE_BLOCKED:` e messaggio human-

readable ("Non è stato caricato il file Procedere comunque?").

- **Client (scheda commessa).** Il pulsante "Avanza" non è più disabilitato dal gate. Al click, se il gate è violato, mostra un `ConfirmDialog` non-destructive con label "Procedi comunque". In conferma chiama `update({ stato, force: true })`.
- **Client (Kanban).** Onerror della mutation: se `err.message` inizia per `D0C_GATE_BLOCKED:`, lo stesso `ConfirmDialog` viene aperto e su conferma rilancia con `force: true`. Errori non-gate restano nel banner inline.
- **La state machine resta invariata.** `force` bypassa SOLO il gate documentale, non la shape delle transizioni.

9.4 Indicatore UI

Sotto l'header della commessa è presente una card con stato del gate:

- Verde se tutti i documenti richiesti sono presenti.
- Ambra se mancano: lista di tipi richiesti con badge "Caricato"/"Mancante" + bottone "Carica file" che preimposta il tipo nel dialog upload.

10. Aperture (Rilievo)

- Una **apertura** è un singolo serramento all'interno di una commessa.
- Campi: `commessaId`, `codice`, `descrizione`, `piano`, `locale`, `tipologia` (`finestra` | `portafinestra` | `porta` | `scorrevole` | `fisso` | `altro`), `larghezza`, `altezza`, `profondita`, `materiale`, `colore`, `vetro`, `noteRilievo`, `criticitaAccesso`, `stato`.
- Stati apertura: `da_rilevare` → `rilevata` → `ordinata` → `consegnata` → `in_posa` → `posata` → `verificata`.
- Le aperture sono visibili nella scheda commessa e nella vista **RilievoDetail** dedicata.
- Una commessa **può** avere zero aperture (es. preventivo iniziale generico).

11. Board Kanban (/kanban)

11.1 Layout

Quattro **fasi**, ognuna con N colonne:

Fase	Colonne
Vendita	Preventivo, Misure Esecutive, Aggiornamento Contratto
Ordine & Produzione	Fatture / Pagamento, Da Ordinare, Produzione
Consegna & Posa	Richiesta Secondo Acconto, Attesa Posa
Chiusura	Finiture / Saldo, Interventi / Regolaz.

Lo stato `archiviata` non compare.

11.2 Card commessa

Mostra: codice, badge priorità, cliente, città, indicatore di consegna (data confermata o indicativa), pulsanti **Indietro** / **Avanza** con etichetta dello stato target. In più (v4):

- **Bordo sinistro 3 px** colorato per priorità (urgente rosso, alta ambra, media blu, bassa grigio).
- **Chip "fermo N gg"** quando la commessa non riceve update da ≥ 5 giorni (ambra) o ≥ 10 (rossa).
- **Blocco prodotti magazzino:** primi 2 prodotti con data consegna corta (✓ verde arrivato, rosso in ritardo) + "+N altri prodotti" (vedi §36).
- **Chip "Da saldare € N"** (rossa) nelle fasi `attesa_posa` / `finiture_saldo` / `interventi_regolazioni` quando il residuo pagamenti è > 0 (vedi §37).

11.2-bis Colonne con overflow

Ogni colonna mostra al massimo **5 card**; oltre compare il toggle tratteggiato "Mostra altre N" / "Mostra meno" per non forzare scroll infiniti.

11.3 Filtri

- Search per codice/cliente/città.
- Filtro priorità (Tutte/Urgente/Alta/Media/Bassa).
- Toggle "Nascondi vuote" / "Mostra vuote".
- Chip per fase (Tutte le fasi / per singola fase).
- Tutte le fasi possono essere collassate.

11.4 KPI bar

Quattro cards: **Attive**, **Urgenti**, **Alte**, **Consegne da confermare** (commesse in `produzione` senza `data-ConsegnaConfermata`).

11.5 Doc gate sul Kanban

Le frecce **Avanza** seguono lo stesso doc gate. Errore `DOC_GATE_BLOCKED`: → ConfirmDialog "Procedi comunque". Errori non-gate → banner inline rosso.

11.6 Esclusioni

- Le commesse soft-archivate non appaiono.

12. Calendario / Planning (/planning)

12.1 Viste

- **Mese** (*default*) — griglia 6×7 (la sesta settimana è renderizzata solo se il mese vi sconfina); oggi evidenziato con pill primary; giorni fuori mese e weekend attenuati; chip evento pieni (colore per tipo, testo bianco) con ora + nome cliente; overflow "+N altri" apre la vista giorno.
- **Settimana** — 7 colonne lun-dom; weekend leggermente attenuati.
- **Giorno** — singola colonna. Ordine switcher: Mese · Settimana · Giorno.

12.2 Tipi di intervento

`rilievo`, `posa`, `assistenza`, `altro`. Colori dedicati per tipo.

12.3 Card intervento (joined info)

Ogni card mostra:

- Ora inizio – ora fine.
- Tipo intervento.
- **Nome + cognome** del cliente (lookup commessa → cliente).
- **Indirizzo** (fallback `intervento.indirizzo` → `commessa.indirizzo` → `cliente.indirizzoLavoro` → `cliente.indirizzo`).
- Note brevi.
- Stato (`pianificato` di default).

12.4 Dialog dettagli appuntamento

In modalità modifica, sopra al form, viene mostrato un blocco di sintesi joined:

- Codice commessa, stato, badge priorità.
- Nome cliente.
- Indirizzo cantiere.
- Telefono (link `tel:`).
- Email (link `mailto:`).
- Squadra assegnata.
- Pulsante "Apri commessa" → naviga alla scheda commessa.

12.5 Auto-fill

Quando si seleziona una commessa in dialog di creazione, l'indirizzo viene auto-popolato dalla commessa (solo se il campo è vuoto, per non sovrascrivere override manuali).

12.6 Drag & drop

- Un intervento può essere trascinato fra giorni nelle viste settimana e mese.
- L'update della data avviene via `interventi.update({ dataPianificata })`.

12.7 Stati intervento

`pianificato`, `in_corso`, `completato`, `sospeso`, `annullato`. L'avvio imposta `dataInizio`; la chiusura imposta `dataFine`. Le entry `annullato` legacy sono **purgate** automaticamente al boot (cleanup nel `persistedStore.onLoad`).

12.8 Link entità

Un intervento può essere collegato a una delle entità: `commessa`, `ticket`, `reclamo`, `rifacimento`. Solo `commessa` è obbligatoria quando il `linkKind === "commessa"`.

12.9 Eventi Google (overlay read-only)

Gli eventi importati dai calendari Google (vedi §38.2) compaiono in tutte e tre le viste, ordinati per ora insieme agli appuntamenti CRM ma **in sola lettura**: stile distinto (badge GOOGLE + lucchetto, bordo sinistro nel colore della sorgente), nessun drag/edit/delete. Il click apre un dialog dettagli (data, orario/tutto il giorno, luogo, calendario di origine). Una legenda sopra la griglia elenca le sorgenti attive.

12.10 Card intervento (restyle v4)

Chip tipo pieno (POSA/RILIEVO/ASSISTENZA/ALTRO) su sfondo tinta + bordo sinistro 4 px nel colore del tipo; ora in mono grassetto. Nel dialog di modifica, accanto al telefono, è presente il bottone **WhatsApp** con messaggio di conferma appuntamento precompilato (vedi §41).

13. Ticket post-vendita (/ticket e contenuti di /reclami)

13.1 Modello

- Campi: commessaId, aperturaId?, oggetto, descrizione, categoria, priorit , stato, assegnatoA?, dataRisoluzione?, esitoIntervento?, apertoBy?.
- Categorie: difetto_prodotto, difetto_posa, regolazione, sostituzione, garanzia, altro.
- Priorit : come per le commesse.

13.2 State machine ticket

aperto → assegnato → in_lavorazione → risolto → chiuso. Disponibile **rollback** di una posizione (clear dataRisoluzione se si esce da risolto/chiuso). Da aperto il rollback   rifiutato.

13.3 Allegati ticket

Vedi §8.7.

14. Reclami e Rifacimenti (/reclami)

Pagina unificata che gestisce due entit  correlate ma distinte.

14.1 Reclamo

- Campi: commessaId, clienteNome, descrizione, responsabile?, stato, dataApertura, dataRisoluzione?, soluzione?.
- Stati: aperto, in_gestione, risolto, chiuso.

14.2 Rifacimento

- Campi: commessaId, clienteNome, descrizione, elemento, fornitoreCoinvolto?, ordineRifacimentoId?, costoStimato?, responsabilit  (interna|esterna), responsabile?, stato, dataApertura, dataChiusura?.

- Stati: `aperto`, `in_gestione`, `in_produzione`, `completato`, `chiuso`.
-

15. Verbali (`/verbale/:interventoId + VerbaleChiusura`)

- Tipo: `chiusura_lavori` | `sopralluogo` | `consegna` (default `chiusura_lavori`).
 - Campi: `interventoId`, `commessaId`, `tipo`, `data`, `noteCliente?`, `noteTecnico?`, `firmaClienteData?`, `firmaTecnicoData?`, `firmaCliente`, `firmaTecnico`, `apertureCompletate`, `apertureTotali`, `anomalieRiscontrate`, `stato` (`bozza`|`firmato`).
 - Lo stato passa a `firmato` quando entrambe le firme sono presenti.
-

16. Anomalie

- Campi: `commessaId`, `aperturaId?`, `interventoId?`, `categoria`, `priorita`, `descrizione`, `risoluzione?`, `stato`, `segnalataBy?`, `risoltaBy?`, `risoltaAt?`.
 - Categorie: `materiale_difettoso`, `misura_errata`, `danno_trasporto`, `difetto_posa`, `problema_accessorio`, `non_conformita`, `altro`.
 - Priorità: `bassa`, `media`, `alta`, `critica`.
 - Stati: `aperta`, `in_gestione`, `risolta`, `chiusa`. Endpoint dedicato `resolve` imposta `stato=risolta`, `risoluzione`, `risoltaAt`.
-

17. Garanzie (`/garanzie` , `direzione-only`)

- Tipi: `prodotto`, `posa`, `accessorio`, `vetro`, `altro`.
 - Campi: `commessaId`, `aperturaId?`, `tipo`, `descrizione`, `fornitore?`, `dataInizio`, `durataMesi`, `dataScadenza`, `stato`, `documentoRif?`, `note?`.
 - `dataScadenza` calcolata server-side da `dataInizio` + `durataMesi`.
 - Stati: `attiva`, `scaduta`, `sospesa`, `revocata`.
 - Statistiche: `totale`, `attive`, `in scadenza` (entro 90 giorni), `scadute`.
-

18. Squadre (`/squadre` , `direzione-only`)

- Campi: `nome`, `caposquadra?`, `telefono?`, `note?`, `attiva`.
 - Le squadre attive appaiono nei dropdown della scheda commessa, dialog intervento, calendario.
 - L'inattivazione (toggle `attiva`) preserva storico ma rimuove la squadra dai picker.
-

19. Fornitori (/fornitori , direzione-only)

19.1 Anagrafica fornitore

- Campi: `ragioneSociale`, `partitaIva`, `indirizzo?`, `citta?`, `telefono?`, `email?`, `categoria`, `referenteCommerciale?`, `scontistica?`, `note?`, `attivo`.
- Categorie: `pvc`, `alluminio`, `vetro`, `ferramenta`, `persiane`, `blindati`, `accessori`, `guarnizioni`, `altro`.

19.2 Ordini fornitore

- Campi: `fornitoreId`, `commessaId`, `codiceOrdine`, `stato`, `dataOrdine`, `dataConsegnaPrevista?`, `dataConsegnaEffettiva?`, `righe[]`, `noteOrdine?`, `noteRicevimento?`, `importoTotale`.
- Stati: `bozza`, `inviato`, `confermato`, `in_transito`, `ricevuto_parziale`, `ricevuto`, `contestato`.
- Riga ordine: `descrizione`, `codiceArticolo?`, `quantita`, `quantitaRicevuta`, `unitaMisura`, `prezzoUnitario?`, `lotto?`, `conforme?`, `noteDifetto?`.
- L'update di stato accetta `righeAggiornate` per registrare la ricezione parziale.

19.3 Listini fornitore

- Campi: `fornitoreId`, `nome`, `versione`, `dataValidita`, `nomeFile`, `tipo` (`pdf|excel|altro`), `note?`.
- I listini sono solo metadati: il file effettivo viene gestito altrove (cartella condivisa o sezione documenti separata).

20. Produzione (/produzione , direzione-only)

Tre sotto-router: `bom` (distinte base), `fasi`, `nc` (non conformità).

20.1 Distinte base (BOM)

- Campi: `commessaId`, `aperturaId`, `stato`, `componenti[]`, `noteValidazione?`, `validataDa?`, `dataValidazione?`.
- Componenti: `tipo` (`profilo|vetro|ferramenta|guarnizione|accessorio`), `descrizione`, `codiceArticolo?`, `fornitoreId?`, `quantita`, `unitaMisura`, `lotto?`, `note?`.
- Stati BOM: `bozza`, `validata`, `in_produzione`, `completata`.
- Validazione consentita solo da `bozza` → `validata`.

20.2 Fasi produzione

- Campi: `commessaId`, `aperturaId?`, `distintaBaseId`, `fase`, `ordine`, `stato`, `operatore?`, `dataInizio?`, `dataFine?`, `checklistItems[]`, `note?`.
- Stati fase: `da_fare`, `in_corso`, `completata`, `non_conforme`.
- Checklist item: `descrizione`, `obbligatorio`, `completato`, `esito?` (`ok|non_conforme`), `note?`. Toggle dedicato `toggleChecklist`.
- `dataInizio` impostata su `in_corso`; `dataFine` su `completata`.

20.3 Non conformità (NC)

- Campi: `commessaId`, `aperturaId?`, `faseProduzioneId?`, `tipo`, `gravita`, `descrizione`, `azione-Correttiva?`, `stato`, `segnalataDa`, `dataApertura`, `dataChiusura?`.
 - Tipi: `materiale_difettoso`, `errore_taglio`, `errore_assemblaggio`, `vetro_rotto`, `ferramenta_errata`, `altro`.
 - Gravità: `lieve`, `media`, `grave`, `bloccante`.
 - Stati: `aperta`, `in_gestione`, `risolta`, `chiusa`. La chiusura imposta `dataChiusura`.
-

21. Preventivatori (/preventivatori)

Pagina hub con cards per azienda. L'accesso è aperto a tutti gli utenti autenticati.

21.1 Fivizzanese — Persiane (/preventivatori/fivizzanese/persiane)

Calcolo step-by-step:

1. **Selezione modello** (catalogo `shared/listini/fivizzanese.ts`).
2. **Quantità persiane standard** (con prezzo base modello).
3. **Quantità persiane con centinatura** (prezzo extra).
4. **Posa** (preset).
5. **Smontaggio vecchie** (opzionale, prezzo per unità).
6. **Promo** quando applicabile:
 - **Promo "RAL 6005 Opaco"**: prezzo aggiornato a **210 €** (precedentemente 200 €).
 - Quando la promo è attiva, viene aggiunto un **ricarico del 50%** sul totale persiane.
7. **IVA** (10% o 22%) — selettore.
8. Output:
 - Riepilogo a video con righe: totale persiane, supplementi, centinatura, ricarico promo, smontaggio, imponibile, IVA, totale lordo.
 - Esportazione **PDF** con `jspdf-autotable`; salvataggio automatico come documento `preventivo` sulla commessa (`keepNome=true`; nome file `"Preventivo {cliente} - Fivizzanese.pdf"`).

21.2 Punto del Serramento — Persiane (/preventivatori/punto-del-serramento/persiane)

Stesso schema più:

- **Sconto** percentuale modificabile, **max 30%** (clamp server-side e client-side).
 - **IVA** (10% o 22%) — selettore.
 - Output PDF analogo con righe `lordo`, `sconto`, `imponibile`, `IVA`, `totale`.
-

22. Archivio (/archivio)

22.1 Soft-archive

- Endpoint `commesse.archive(id)`: imposta `archivedAt = ISO now`.

- Endpoint `commesse.restore(id)` : imposta `archivedAt = null`.
- Lo stato (`stato`) e i collegamenti (file, aperture, interventi, prodotti) **non vengono toccati**. Restore = la commessa torna esattamente com'era.

22.2 Visibilità

- Le commesse soft-archivate sono escluse da:
 - `commesse.list` (scope default `exclude`).
 - `commesse.stats` e `commesse.byPriorita`.
 - Board Kanban.
 - Dashboard.
 - Notifiche proattive (vedi §27).
 - Classifica venditori.
- Restano accessibili a `/archivio` (scope `only`) e tramite link diretto `/commesse/:id`.

22.3 Pagina Archivio

- Lista con search per codice/cliente/città/indirizzo.
- Per ogni riga: codice, stato originale (badge), priorità, cliente, indirizzo, data apertura, data archiviazione, pulsanti **Ripristina** e **Apri scheda**.
- Conferma esplicita prima del ripristino.

22.4 Pulsanti nella scheda commessa

- **Archivia** (se non archiviata) — conferma non-destructive.
- **Ripristina** (se archiviata).
- Banner di stato "Commessa archiviata" in alto quando applicabile.

23. Classifica venditori (`/classifica`)

23.1 Regola di conteggio

Per ogni utente con ruolo `commerciale` e `attivo === true`, conta le commesse che soddisfano TUTTE queste condizioni:

1. `assegnatoA === utente.id`.
2. `archivedAt === null` (no soft-archive).
3. `stato ∈ {misure_esecutive, aggiornamento_contratto, fatture_pagamento, da_ordinare, produzione, ordini_ultimazione, attesa_posa, finiture_saldo, interventi_regolazioni}` — esclusi `preventivo` (non ancora "vera" commessa) e `archiviata` (chiusura).

23.2 Ordinamento e ranking

- Ordine: count desc, tie-break alfabetico (`cognome nome`).
- Rank **competition (1224)**: due pari-merito al primo posto → il successivo è 3°.

23.3 Podio "intelligente"

- Il podio mostra fino a **3 score-tier** (gradini), non 3 persone:
 - Tutti i pari-merito condividono lo stesso gradino.
 - Se due venditori sono primi a pari merito, il gradino di argento può essere vuoto e il successivo è terzo con rank 3.
- Se **tutti** hanno lo stesso punteggio → un solo gradino oro con banner *"PARI MERITO TOTALE — SIETE TUTTI PRIMI"*.
- Layout visuale: 2 — 1 — 3 (vincitore al centro), gradino tallone più alto in oro.

23.4 Interattività e "battute"

- **Coriandoli CSS** all'apertura della classifica, replay su click.
- **Banner Commento della giuria** (amber, prominente, rotazione automatica ogni 5 s + click).
- **Bolla citazione vincitore** (rotazione click).
- **Click su qualunque atleta** → coriandoli + toast spiritoso ("X scalda i motori!", "Occhio, X è in modalità squalo", ...).
- **Count-up** animato da 0 al valore finale per ogni numero commesse.
- **Tag "Cucchiaino di legno"** all'ultimo, badge "-N dal 1°" per gli inseguitori, fun-fact rotante nel footer.
- Zero dipendenze esterne per gli effetti (CSS keyframes + sonner già presente).

23.5 Empty state

- Nessun utente `commerciale` attivo → "Podio vuoto, eco assordante. Aggiungi utenti con ruolo «commerciale» dalla pagina Utenti".

24. Utenti (`/utenti` , direzione-only)

24.1 Funzionalità

- Lista utenti con filtro per ruolo + search testuale.
- Creazione, modifica, eliminazione (tutte `adminProcedure`).
- Multi-ruolo (1–3 ruoli per utente).
- Toggle `attivo/inattivo` .
- Password gestita solo lato server (hashing script, vedi §3.3). Il campo `hasPassword` (bool) è restituito al client al posto del contenuto.

24.2 Gating

- Rotta `/utenti` protetta da `<RequireDirezioe>` .
- Sidebar mostra la voce **Utenti** solo a utenti `direzione` .

24.3 Email

- Univocità email enforced lato server (`utenti.create` rifiuta duplicati case-insensitive).

25. Notifiche proattive (dropdown header) — v2 personalizzate

25.1 Personalizzazione

Le notifiche sono calcolate on-demand **per l'utente corrente**: "owner" = `commessa.assegnatoA === userId` (fallback legacy `createdBy` quando `assegnatoA` è nullo). La `direzione` vede anche le notifiche di scope sede. Tutto è filtrato sulla sede attiva.

25.2 Fonti

#	Fonte	Destinatari	Severità	Note sull'id
1	Priority aging — commessa ferma oltre soglia (bassa 7 gg, media 5, alta 3, urgente 1)	owner	per priorità	embedde <code>updatedAt</code> : "segna letta" tace finché la commessa non si muove di nuovo
2	Daily reminder su stati bottleneck (<code>aggiornamento_contratto</code> , <code>fatture_pagamento</code> , <code>da_ordinare</code>)	owner	escalation per età (≥5 urgent, ≥3 warning)	embedde la data: letta oggi, rispunta domani
3	Stato → ruolo (<code>da_ordinare</code> →ordini, <code>misure_esecutive</code> →tecnico_rilievi, <code>fatture_pagamento</code> / <code>finiture_saldo</code> →amministrazione)	utenti col ruolo	info	id = (commessa, stato): letta finché lo stato non cambia
4	Consegna da confermare — <code>produzione</code> senza <code>dataConsegnaConfermata</code>	owner + direzione	warning	id stabile per commessa
4b	Saldo residuo nelle fasi finali (<code>attesa_posa</code> , <code>finiture_saldo</code> , <code>interventi_regolazioni</code>) con <code>importoTotale</code> impostato e residuo > 0	owner + direzione + amministrazione	warning	id embedde il residuo: un incasso parziale ri-notifica
5	Garanzie scadute (urgent) o in scadenza ≤30 gg (warning)	amministrazione + direzione	urgent/warning	id per garanzia
6	Ticket aperti/assegnati su commesse possedute	owner + direzione	da priorità ticket	id ruota con lo stato: riapertura ri-notifica
7	Interventi di oggi/domani senza squadra	owner (direzione per i non collegati)	warning oggi / info domani	id per (intervento, data)

25.3 Stato lettura persistito

- Store `notifiche_read`: una riga per utente `{ userId, readIds[] }` (cap 800 id, FIFO).
- `notifiche.markRead({ids})` e `notifiche.markAllRead()`.
- `notifiche.list` ritorna `read: boolean`; ordinamento: non lette prima, poi severità (urgent→warning→info), poi recency. Cap 100.
- `notifiche.count` = **solo non lette** (badge campanella: rosso se contiene urgenti, ambra altrimenti).
- Ogni notifica porta un campo `link` (commessa, `/ticket`, `/garanzie`, `/planning`): il click nel drop-down segna letta e naviga.

25.4 Dropdown

Header con conteggio "N da leggere — personalizzate per te" + bottone "Segna tutte lette". Icone per tipo (camion=consegna, ticket, scudo=garanzia, calendario=intervento). Le lette sono attenuate in fondo; pallino primary sulle non lette.

25.5 Esclusioni

Come v3: niente notifiche per commesse `archiviata` o soft-archivate.

26. Dashboard (/)

26.1 Header personale

"Ciao {nome} — ecco la tua giornata" + data lunga it-IT.

26.2 "Da fare oggi" — feed azioni personalizzato

- **Scope**: direzione vede tutta la sede (etichetta "Tutta la sede"); gli altri solo le proprie commesse (`assegnatoA`, fallback `createdBy`) — etichetta "Le tue attività".
- Fonti, ordinate per urgenza e cap a 8 voci:
 1. Interventi di **oggi senza squadra** (CTA "Apri calendario").
 2. Commesse **urgenti** / ticket urgenti / garanzie scadute.
 3. **Da incassare € N** — residuo pagamenti nelle fasi finali.
 4. **Consegne da confermare** (CTA "Conferma consegna").
 5. Ticket aperti sulle proprie commesse.
 6. Garanzie in scadenza 30 gg (direzione/amministrazione).
- Stato vuoto esplicito: "Niente da fare per ora ..." con icona verde (la card non sparisce).

26.3 KPI principali

Cards Commesse attive, Urgenze, Consegne da confermare, Ticket aperti (+ Interventi settimana). Zero = card "spenta" non cliccabile; >0 = accent bar + navigazione alla lista filtrata. Polling live.

26.4 Calendario settimanale

Slot 7 giorni con eventi per tipo (filtri per calendario) + navigazione settimana.

27. Integrazioni esterne (/integrazioni = Impostazioni)

La pagina Impostazioni ospita l'hub **Gestione** (direzione-only: Fornitori, Produzione, Squadre, Garanzie, Preventivatori) e le card integrazioni:

27.1 Backup notturno su Google Drive

Vedi §39. Card con stato collegamento, ultimo backup, prossima esecuzione, "Esegui ora", collega/scollega account.

27.2 Fatture in Cloud → Clienti

Vedi §40. Card con token (mascherato), selettore azienda, switch abilitazione, "Sincronizza ora", esito ultimo sync.

27.3 Mostra Google nel calendario CRM (import)

Vedi §38.2. Gestione sorgenti: nome + indirizzo iCal segreto (mascherato in lista), colore auto-assegnato, toggle mostra/nascondi, stato sync, "Aggiorna", istruzioni passo-passo.

27.4 Pubblica il calendario CRM su Google (export)

Vedi §38.1. Elenco feed copiabili (Tutti/Rilievi/Pose/Assistenza/Altro) + "Rigenera token" con avviso di revoca.

27.5 Microsoft To Do

Card presente ma **mock** (non ancora collegata a Graph API). Storicamente il flusso operativo viveva su liste To Do; la migrazione del 15/07/2026 (vedi §43) le ha importate nel CRM.

27.6 Notifiche owner / Google Maps

Invariati da v3: `system.notifyOwner` (Manus Notification Service, admin-only) e componente `<Map/>` via proxy interno.

27.7 Roadmap: Antenore (Wnd/Oknoplast)

Integrazione richiesta al fornitore (import automatico clienti/preventivi/commesse/ordini creati sul loro CRM). In attesa di risposta tecnica sugli endpoint disponibili; prevista sync unidirezionale Antenore → Ruffino Ops con dedup su id.

28. Persistenza

28.1 KV store

- Tabella `kv_store`(key text primary key, data jsonb, updated_at timestamptz).
- Ogni router business possiede una o più "raccolte" persistite: `clienti`, `commesse`, `aperture`, `interventi`, `anomalie`, `tickets`, `ticket_allegati`, `squadre`, `garanzie`, `verbal`i, `fornitori`,

fornitori_ordini, fornitori_listini, produzione_distinte, produzione_fasi, produzione_nc, preventivi_documenti, reclami, rifacimenti, timeline_steps, utenti, sedi, calendar_tokens, external_calendars, notifiche_read, backup_config, backup_log, backup_oauth, fic_config, magazzino_prodotti.

Il refresh token Google del backup è inoltre **specchiato su file** (data/backup-oauth.json , mode 600, gi-gnored) così i riavvii senza DATABASE_URL non scollegano Drive; la riga DB, quando presente, ha precedenza.

28.2 Lifecycle

- **Bootstrap.** All'avvio bootstrapAll() legge ogni raccolta. Tre stati possibili per la singola key:
 - firstBoot = true : nessuna row presente → callback onLoad può popolare seed.
 - firstBoot = false, loaded = true : row presente, dati ripristinati con revive Date.
 - firstBoot = false, loaded = false : errore transiente; **i save sono bloccati** finché la background recovery non riesce a leggere lo stato dal DB.
- **Save.** Debounciato 200 ms; usa sql.json() per evitare doppia encoding. Retry exponential backoff su errori transienti (EAI_AGAIN, ENOTFOUND, ECONNREFUSED, ECONNRESET, ETIMEDOUT, ENETUNREACH, EHOSTUNREACH, EPIPE).
- **Shutdown.** Handler SIGTERM / SIGINT chiamano flushAll() per non perdere mutazioni in flight, poi chiudono la pool.

28.3 Recovery

- Se ensureSchema o load falliscono dopo i retry → backgroundRecover (max 30 tentativi a intervalli di 5 s) finché ogni raccolta non risulta loaded = true . Nessun seed viene applicato in caso di fallimento (per non sovrascrivere dati reali con un seed accidentale).

28.4 Legacy double-encoded payload

- Riconosciuto a load: se data è stringa JSON, viene parsata e schedulata una riscrittura corretta.

28.5 Vincoli su Postgres

- Connessione Postgres-js: max 5 connessioni, idle_timeout=20s , connect_timeout=30s (Railway internal DNS warmup).
- SSL automatico quando l'URL contiene sslmode=require o l'host Railway.

29. UI/UX e componenti

29.1 Sistema

- Tailwind + shadcn/ui (Button, Card, Badge, Dialog, AlertDialog, Avatar, Input, Label, Select, Switch, Tabs, Tooltip, DropdownMenu).
- Icone lucide-react.
- Toast: sonner.
- Componente <SearchSelect> riutilizzato per dropdown ricercabili (utenti, squadre, fornitori, ecc.).

- Componente `<ConfirmDialog>` riutilizzato per: cancellazioni, archiviazioni, "Procedi comunque" del doc gate, ripristino archivio.
- Componente `<FilePreviewDialog>` per preview PDF/immagini.

29.2 Sidebar

- Voci dinamiche; quelle con `direzioneOnly` filtrate per ruolo.
- Resizable (larghezza personalizzabile da utente).
- Avatar utente in basso con menù: nome, email, Esci.

29.3 Mobile

- Layout responsivo. Sidebar collassata in modalità mobile; nasconde scritte tenendo solo icone.

29.4 Empty states

- Tutte le pagine principali hanno empty state esplicito con istruzioni sul prossimo passo.
-

30. Errori e telemetria

30.1 Convenzioni errori

- Gli errori non-tRPC sono ritornati come `Error` con messaggio human-readable in italiano.
- Errori "marker" usano un **prefisso** come `DOC_GATE_BLOCKED:` che il client identifica per offrire UI dedicata.

30.2 Logging

- Tutto il logging della persistenza passa da `console.log/warn/error` con prefisso `[persistence]`.
 - I save e i load mostrano sempre il conteggio degli elementi per raccolta.
-

31. Roadmap aperta (lavori noti)

31.1 Sicurezza

- **Rotazione credenziali** committate nello storico Git + **purge history** (`BFG/git filter-repo`).
- CSP (Content Security Policy) tarata su Vite, blob preview, Maps proxy.

31.2 Ottimizzazione

- **Per-file storage** dei documenti: oggi ogni upload riscrive tutta la JSONB della raccolta `preventivi_documents`. Migrazione consigliata verso una row kv per documento o, meglio, una tabella `documents` con colonna `bytea` /object storage esterno. Richiede backup DB prima di toccare i dati.
- Aggregato dashboard in un unico endpoint per ridurre il fan-out lato client.

31.3 UX

- Drag&drop diretto sulle colonne del Kanban (oggi solo bottoni avanza/indietro).
- Confetti hardware-accelerati (opzionale).

31.4 Integrazioni

- **Fatture in Cloud OAuth** (app 21541): flusso authorize + refresh token in sostituzione del token manuale (credenziali già emesse, implementazione pianificata).
 - **Antenore (Wnd/Oknoplast)**: connettore import clienti/preventivi/ordini — in attesa di specifiche dal fornitore.
 - Esportazione CSV/Excel commesse, clienti, anomalie.
 - Push notifications mobile.
 - UI di restore dal backup Drive.
-

32. Glossario

- **Commessa**. Progetto specifico di vendita+installazione per un cliente.
 - **Apertura**. Singolo serramento (finestra, porta, ...) all'interno di una commessa.
 - **Stato**. Posizione corrente della commessa nella state machine.
 - **Soft-archive**. Stato secondario, ortogonale allo stato del workflow: la commessa è nascosta dalle viste operative ma tutto è preservato.
 - **Doc gate**. Vincolo per cui un avanzamento di stato richiede l'upload di documenti previsti per lo stato corrente.
 - **Bypass**. Conferma esplicita dell'operatore tramite `force: true` per superare il doc gate.
 - **Tier (classifica)**. Un gruppo di venditori con lo stesso conteggio commesse; condividono il rank.
 - **Pari merito**. Più persone con lo stesso conteggio. Condividono il rank e il gradino del podio.
 - **Indirizzo di residenza**. Indirizzo fiscale del cliente (uso amministrativo).
 - **Indirizzo di lavoro**. Indirizzo del cantiere (uso operativo per commesse, calendario, mappe).
 - **Tipo detrazione**. Modalità fiscale richiesta dal cliente: `ecobonus` o `ristrutturazione`.
-

33. Cronologia significativa

- v3.0 — Riallineamento completo del PRD al codice corrente: dual address, tipoDetrazione, dataConsegnaIndicativa, soft-archive, Archivio, Classifica venditori, doc-gate con bypass, hardening sicurezza (scrypt, JWT fail-hard, mimeType allowlist, rate-limit login, security headers, session eviction, logout server-side), assegnazione utente modificabile, preventivatori Fivizzanese e Punto del Serramento, Planning con joined info.
- **v4.0 (16/07/2026)** — Multi-sede con isolamento completo; redesign UI (board v2, calendario mese-default con chip pieni, timeline note post-it, dashboard personalizzata); Magazzino (tile+popup, ordini, lead time, dropdown fornitori); registro acconti; notifiche v2 con stato lettura; sincronizzazione Google Calendar export+import; backup notturno Google Drive (OAuth drive.file); Fatture in Cloud sync clienti; WhatsApp deep link; scheda cliente PDF; hardening (scrypt versionato, CSRF, SSRF guard, trust proxy, mascheramento segreti); migrazione dati 2026 (§43).

- v3.0 — Riallineamento completo del PRD al codice corrente: dual address, tipoDetrazione, dataConsegnaIndicativa, soft-archive, Archivio, Classifica venditori, doc-gate con bypass, hardening sicurezza (scrypt, JWT fail-hard, mimeType allowlist, rate-limit login, security headers, session eviction, logout server-side), assegnazione utente modificabile, preventivatori Fivizzanese e Punto del Serramento, Planning con joined info.
 - v2.x — Versione precedente (PDF allegato in repo come riferimento storico).
 - v1 — Documento originale.
-

Parte II — Moduli introdotti dopo la v3

34. Multi-sede

34.1 Modello

- Store `sedi`: { `id`, `nome`, `indirizzo?`, `citta?`, `attiva` } (seed prima sede "La Spezia").
- Ogni entità business porta `sedeId` (backfill = 1 per i record pre-esistenti).
- Gli utenti hanno `sediIds`: `number[]` (accesso multi-sede).

34.2 Risoluzione della sede attiva

- Cookie/claim `active_sede` → `ctx.sedeId` su ogni richiesta tRPC.
- **SedeSwitcher** nella sidebar (in alto): cambia la sede attiva; tutte le query si rifanno sul nuovo scope.

34.3 Isolamento

- Ogni `list` filtra per `sedeId`; ogni mutation su entità esistente passa da `assertSedeScope` (404 anche in caso di id valido di altra sede — nessun information leak).
- Entità figlie (documenti, allegati, timeline, magazzino) ereditano lo scope dalla commessa/ticket padre.
- Pagina `/sedi` (direzione-only) per creare/gestire sedi; assegnazione `sediIds` dalla pagina Utenti.

35. Timeline ordine (scheda commessa)

35.1 Struttura

18 step fissi per commessa (store `timeline_steps`, creati lazy alla prima lettura), raggruppati nelle 4 fasi del board: Rilievo Misure, Firma Contratto, Fatturazione, Invio Fattura, 1° Acconto, Ordine Merce, Conferma Ordine, Acconto Fornitore, Data Spedizione Prevista, Pagamento Merce Pronta, 2° Acconto, Data Consegna Merce, Appuntamento Posa, Lista Merce Posata, DDT Posa, Finiture, Saldo, Recensione.

35.2 Interazione

- Barra avanzamento ($N/18 + \%$). Fasi collapsabili; **la fase con lo step corrente e quelle contenenti note si aprono da sole.**

- Step corrente evidenziato (sfondo primary tenue) con bottone **Completa** one-click (data odierna). Dialog di modifica per utente esecutore (SearchSelect) + note; step completato riapribile.
- Campi step: stato (da_fare|in_corso|completato), dataCompletamento, utente, note, allegato? .

35.3 Note come cittadino di prima classe

- Le note renderizzano come **post-it ambra** (icona + 12 px, `whitespace-pre-line` : le note multiriga migrate dai To Do conservano a-capo e separatori "— — —").
- L'header di ogni fase mostra un badge ambra "📝 N" con il conteggio note.
- Le date di completamento sono formattate it-IT.

36. Magazzino (/magazzino)

36.1 Scope

Prodotti fisici in arrivo/da magazzino **per commessa**. Eleggibili solo commesse **da produzione in poi** (incluso; escluse archiviate) — gate applicato sia lato server (create) sia nel filtro pagina.

36.2 Modello (magazzino_prodotti)

```
{ id, sedeId, commessaId, nome, quantita, fornitore?, numeroOrdine?, dataOrdine?, dataConsegna?, arrivato, note?, createdAt, updatedAt }
```

- **Fornitore** da dropdown fisso aziendale: Wnd, Oknoplast, Alias, Pail, Primed, HenryGlass, Palmieri, Errecci, Fivizzanese, Oskura, Korus, Punto del Serramento, Kopern, Citea, Cerrato, Brianzatende, Seraplastic, St Scale, Sharknet (valori legacy liberi restano selezionabili).
- **Lead time** = giorni `dataOrdine → dataConsegna` , mostrato per prodotto (🕒 N gg) e come **KPI medio** sugli arrivati.
- Cascade: eliminando una commessa si eliminano i suoi prodotti.

36.3 UI

- **KPI**: Prodotti, In arrivo, Arrivati, Lead time medio.
- **Strip "Prossime consegne"**: le 5 consegne pendenti più vicine cross-commessa (rosse se scadute); il click apre il popup della commessa.
- **Ricerca + filtri**: Tutte / Con prodotti / In arrivo / In ritardo / Arrivati.
- **Griglia di tile** (3 per riga desktop): codice, StatoChip, cliente (17 px bold), città, primi 2 prodotti colorati per stato (✓ verde arrivato, rosso in ritardo con data corta, grigio in arrivo), "+N altri prodotti", badge "N/M arrivati". Bordo rosso (ritardi) / verde (tutto arrivato). Ordinamento per urgenza (ritardi → consegna più vicina).
- **Popup dettaglio** (click sul tile): header con codice/cliente/stato/città + "Apri commessa" + badge; righe prodotto **tutte editabili inline** (quantità, fornitore, n° ordine, date, switch arrivato, nota ghost click-to-edit con blur-save); form "Aggiungi prodotto" su due righe.

36.4 Board

Le card del Kanban mostrano il blocco prodotti (vedi §11.2).

37. Pagamenti — registro acconti

37.1 Modello

Su ogni commessa: `importoTotale` (pattuito) + `pagamenti[]` embedded: { `id`, `importo`, `data?`, `metodo?`, `note?`, `createdAt` } con `metodo` ∈ `bonifico` | `contanti` | `assegno` | `pos` | `finanziamento` | `altro`.

- `importoIncassato` è **sempre ricalcolato dal server** come somma del registro (`add/removePagamento`) → board, dashboard e notifiche coerenti senza duplicare logica.
- Backfill: record legacy con incassato secco → unico acconto "Importo importato".

37.2 Card "Pagamenti" (scheda commessa)

- Totale pattuito editabile inline (blur-save), barra "% incassato · € N", **Residuo** grande (ambra finché > 0, verde a saldo).
- Registro acconti ordinato per data: data, importo bold, badge metodo, nota, cestino.
- **Chips rapide 50% / 40% / 10%** del totale (il piano di pagamento tipico), cappate al residuo: un click precompila l'importo nel form di registrazione (data default oggi).
- Accent warning sulla card finché c'è residuo.

37.3 Propagazione

- Board: chip "Da saldare € N" nelle fasi finali (§11.2).
- Dashboard: voce "Da incassare € N" nel feed personalizzato (§26.2).
- Notifiche: fonte 4b (§25.2) — l'id embedde il residuo, un incasso parziale ri-notifica il nuovo valore.

38. Sincronizzazione Google Calendar

38.1 Export (CRM → Google) — feed iCal per sede

- Ogni sede ha un **token segreto** (store `calendar_tokens`, rigenerabile: la rigenerazione revoca tutte le iscrizioni).
- Endpoint anonimo `GET /api/ics/:token/:feed.ics` (il token è il bearer) con cache 5 min. Feed: `tutti`, `rilievo`, `posa`, `assistenza`, `altro`.
- ICS RFC 5545 con VTIMEZONE Europe/Rome; titolo = tipo + cliente; location = indirizzo lavoro.
- L'operatore aggiunge il feed in Google Calendar via "Altri calendari → Da URL"; Google lo aggiorna periodicamente (sola lettura lato Google).

38.2 Import (Google → CRM) — overlay in Planning

- Sorgenti per sede (store `external_calendars`): nome, **indirizzo iCal segreto** di Google ("Impostazioni → Integra calendario"), colore da palette, attivo.
- Fetch **server-side** (evita CORS) con cache 10 min, follow redirect, mantiene lo stale su errore; `webcal://` normalizzato a https; **SSRF guard** (§3.7).
- Parser ICS interno: unfolding, unescape, DATE/DATE-TIME/UTC, TZID→Europe/Rome; **espansore RRU-LE** limitato alla finestra richiesta (DAILY/WEEKLY/MONTHLY/YEARLY, INTERVAL, COUNT, UNTIL, BYDAY, EXDATE, cap iterazioni).

- Gli eventi appaiono read-only nel Planning (§12.9).

39. Backup notturno su Google Drive

39.1 Contenuto e struttura

Ogni notte alle **00:00 Europe/Rome** (o manualmente) viene generato uno snapshot datato:

```
Backup CRM YYYY-MM-DD/
  database/<store>.json      ← dump integrale di ogni raccolta (utenti SENZA password)
  Sede <nome>/
    Utenti.json              ← utenti della sede (sanificati)
    Clienti/<Cognome Nome (CL-id)>/
      Scheda cliente.pdf      ← generata server-side (gemella della scheda in app)
      cliente.json
    Commesse/<CODICE>/
      commessa.json
      Preventivi e contratti/... ← file caricati, raggruppati per tipo
      Misure/... · Fatture e pagamenti/... · Ordini/... · DDT/... · Foto e altro/...
      Ticket <id>/...         ← allegati ticket
    Commesse senza cliente/<CODICE>/...
```

39.2 Collegamento Google (account personale)

- I service account NON possono scrivere su My Drive personale (Google One) → modalità primaria **OAuth utente**: la direzione collega l'account aziendale (una volta) con scope **drive.file** (l'app vede solo i file che crea).
- Flusso: `backup.oauthStartUrl` (adminProcedure, state monouso anti-CSRF) → authorize Google → callback anonima `/api/oauth/gdrive/callback` → refresh token salvato (store + specchio su file, §28.1).
- Al primo backup l'app crea la cartella **"Backup CRM Ruffino"** nel My Drive collegato; l'operatore può spostarla/condividerla ovunque (tracciata per id — attualmente dentro la cartella condivisa aziendale).
- Fallback: senza credenziali il backup viene comunque scritto su disco server (`backups/` , `gitignored`). Modalità service account mantenuta nel codice per un eventuale passaggio a Workspace/Shared Drive.

39.3 API & UI

- Drive v3 via REST puro (JWT/token con crypto nativo — zero dipendenze npm); find-or-create cartelle con cache per run; upload multipart; `supportsAllDrives` .
- Router `backup` (direzione): `status` , `log` (ultimi 60 run), `runNow` , `updateConfig` , `oauthStartUrl` , `disconnectOAuth` , `checkRoot` .
- Card in Impostazioni: stato ("Drive collegato" + email account), ultimo backup (esito/file/MB), countdown prossimo run, "Esegui ora", collega/scollega, istruzioni una-tantum.
- Single-flight guard (un backup alla volta); log persistito.

40. Fatture in Cloud → Clienti

40.1 Funzione

Ogni 6 ore (se abilitato) o su "Sincronizza ora", il CRM legge le **fatture emesse dell'anno corrente** e **crea i clienti mancanti** (mai tocca le fatture, mai aggiorna clienti esistenti).

40.2 Logica di creazione

- Dedup su denominazione normalizzata contro i clienti esistenti.
- Persone: split cognome/nome **validato col codice fiscale** (consonanti del cognome vs primi 3 caratteri del CF; supporta cognomi composti e denominazioni invertite). CF valido salvato sul cliente.
- Aziende (P.IVA presente o ragione sociale riconosciuta): denominazione completa in `cognome`, tipo `azienda` (o `condominio`).
- I clienti creati vanno sulla sede primaria.

40.3 Config & stato

Store `fic_config`: token API (mascherato in status), companyId (picker "Trova azienda" via `/user/companies`), enabled, ultimo esito. Router `fattureInCloud` (direzione): `status`, `saveConfig`, `companies`, `syncNow`. **Roadmap**: passaggio a OAuth app (credenziali già emesse) con refresh automatico (§31.4).

41. WhatsApp (deep link)

- Helper `lib/whatsapp.ts`: normalizzazione numeri italiani → `wa.me/39...` con messaggio precompilato; firma aziendale "Ruffino Group — tel. 0187 872687".
- Bottone verde accanto al telefono in: **scheda cliente** (messaggio generico pratica), **scheda commessa** (messaggio con codice commessa), **dialog appuntamento del Planning** (conferma appuntamento con tipo, data e ora).
- Nessun invio automatico: si apre WhatsApp con il testo pronto, l'operatore preme invia.

42. Scheda cliente PDF

- Bottone "Scheda PDF" nella scheda cliente → A4 via jsPDF/autotable: anagrafica completa (tipo, contatti, CF/P.IVA, doppio indirizzo, detrazione, pratica edilizia, assegnatario), note, e tabelle Commesse / Appuntamenti / Ticket / Garanzie con section header e page-break safe.
- La stessa scheda è generata **server-side** per ogni cliente nel backup notturno (§39.1).

43. Migrazione dati 2026 (eseguita il 15/07/2026)

Operazione una-tantum documentata per riferimento storico:

1. **Clienti**: 101 clienti unici estratti dal riepilogo fatture 2026 di Fatture in Cloud (113 righe); 73 creati, 28 già presenti. Split cognome/nome validato via CF; aziende riconosciute da P.IVA.
2. **Arricchimento**: incrocio con l'export anagrafico completo (846 record) → 72 clienti completati con indirizzo/città/CAP/telefono/email/CF/P.IVA; 1 inversione nome corretta via CF.

3. **Commesse + stati + note:** dalle 7 liste Microsoft To Do di reparto (Misure Esecutive, Aggiornamento Contratto, Ordini, Produzione, Attesa Posa, Finiture a saldo, Interventi/Regolazioni) → 43 commesse create, 23 riusate, 62 stati portati alla fase corretta, 71 note operative scritte negli step timeline corrispondenti (Firma Contratto / Ordine Merce / Data Spedizione / Appuntamento Posa / Finiture).
4. **Dedup:** merge di 5 doppioni (typo, nomi invertiti, coniugi cointestatari) con fusione di note timeline e stati; 3 coppie legittime (persona + propria azienda, conviventi) lasciate distinte.
5. Le righe To Do di clienti non-2026 (fatture 2023-25) sono state escluse deliberatamente.